Hawassa University
Institute of Technology

Faculty of Informatics
Department of Computer Science

Advanced Programming (CoSc3052) – Test1
Computer Science 3rd Year Regular – (Section 3)
Instructor: Elias Jarso

Date: Nov 3, 2025GC
Time Allowed: 2:00Hrs
Max Weight: 20%

NAME: _____ ID:_____ Sign:_____

General Instructions:

1. Attempt all the questions! Partial attempts are better than leaving questions blank.
2. A coding attempt is MANDATORY for every programming question.
3. Write CLEAR and WELL-FORMED answers! Illegible, unstructured, or ambiguous answers will receive zero points.
4. Time management is your responsibility. Allocate enough time for debugging and revising your answers.
5. Do NOT copy from other students or external sources. Any form of cheating or academic dishonesty will result in cancellation of your exam results.
6. Respect exam conduct rules. Phones, laptops, smart devices, and communication tools are strictly prohibited.

Section I: Multiple Choice Questions (33 Marks)

Instructions: Select the single best answer for each question. Each question is worth 1 mark.

Architecture & Lifecycle of JavaFx

1. Which statement **best encapsulates** the core **architectural weakness** of AWT that Swing was designed to solve?
   a) AWT's reliance on native OS components led to a limited, "lowest common denominator" set of UI controls.
   b) AWT's pure Java rendering was too slow on older hardware.
   c) AWT did not have a layout management system.
   d) AWT applications were not truly cross-platform.
2. JavaFX's modern architecture is **primarily characterized** by its:
   a) Reversion to AWT's heavyweight, native-peer model.
   b) Use of a Scene Graph and a GPU-accelerated rendering pipeline.
   c) Tight integration with the Swing component library.
   d) Reliance on a single-threaded model for both UI and background tasks.
3. In the JavaFX application lifecycle, the **init()** method is the **designated place for**:
   a) Constructing the UI, as it runs on the JavaFX Application Thread.
   b) Performing long-running background tasks before the UI appears.
   c) Initializing the primary Stage and showing it.
   d) Non-GUI initialization, as it runs on the main Java thread, not the JavaFX Application Thread.
4. The **stop()** method is guaranteed to be called on which thread?
   a) The main Java thread.
   b) The JavaFX Application Thread.
   c) A dedicated shutdown hook thread.
   d) A new background thread created for cleanup.
5. The **fundamental purpose** of the **Scene Graph** in JavaFX is to:
   a) Manage the application's lifecycle from *init()* to *stop()*.
   b) Represent the hierarchical tree of all visual elements, defining their rendering order and event propagation path.
   c) Store and manage CSS styles for the application.
   d) Handle background threading and task management.

6. Which statement **accurately describes** the relationship between Stage, Scene, and a root Node?
   a) A Scene contains a Stage, which in turn contains a root Node.
   b) A Node is the root, containing a Stage which holds the Scene.
   c) A Stage is the window, which contains a Scene, and the Scene must contain one root Node.
   d) Stage and Scene are interchangeable terms for the main application window.
7. If a long-running (5-second) database query is executed directly inside a Button's setOnAction handler, what will be the user's experience?
   a) The UI will remain responsive while the query runs in the background.
   b) The UI will freeze and become completely unresponsive until the handler method completes.
   c) Only the clicked button will become disabled; other UI elements will work.
   d) The query will automatically be cancelled after 2 seconds to prevent UI freezes.
8. The Platform.runLater() method is essential for:
   a) Running a task with a delay.
   b) Ensuring a piece of code that modifies the UI is executed safely on the JavaFX Application Thread.
   c) Pausing the JavaFX Application Thread for a short period.
   d) Executing code before the start() method is called.

LAYOUT PANES

9. To create a layout where a central component grows and shrinks while components at the top, bottom, left, and right remain fixed, the most **appropriate root pane** is:
   a) GridPane b) AnchorPane c) BorderPane d) FlowPane
10. Which layout pane arranges its children in a grid and allows a child to span multiple columns or rows?
   a) TilePane b) GridPane c) HBox d) StackPane
11. To ensure a TextField within a GridPane's second column expands horizontally to fill all available space, the most robust method is:
   a) textField.setPrefWidth(Double.MAX_VALUE);
   b) GridPane.setHgrow(textField, Priority.ALWAYS);
   c) textField.setStyle("-fx-width: 100%;");
   d) GridPane.setColumnSpan(textField, 2);
12. A key feature of FlowPane that distinguishes it from HBox is its ability to:
   a) Center its children vertically.
   b) Set spacing between children.
   c) Add padding around its content.
   d) Wrap its children to the next row/column when space runs out.
13. You need to overlay a semi-transparent "**Loading...**" Label on top of an existing ImageView (i.e. *You need to place a* 'Loading…' Label *on top of an* ImageView *so that both occupy the same space*). Which layout pane is specifically designed for this kind of layering?
   a) BorderPane b) StackPane c) AnchorPane d) VBox
14. To create a UI where a Button is permanently "docked" 10 pixels from the bottom and 10 pixels from the right edge of its parent pane, regardless of window size, you should use:
   a) An HBox with right-aligned content.
   b) A BorderPane with the button in the BOTTOM region.
   c) An AnchorPane with bottom and right anchors set to 10.0.
   d) A GridPane with carefully configured RowConstraints.
15. To correctly place three buttons horizontally within the single TOP region of a BorderPane, you must:
   a) Call borderPane.setTop() three times, once for each button.
   b) Nest the three buttons within an HBox and then set that HBox as the TOP node.
   c) Use an AnchorPane as the root and set top anchors for all three buttons.
   d) This is not possible; the TOP region can only hold one child Node.

16. Assume you have already created three buttons: `btn1`, `btn2`, and `btn3`. You now want to add them to a JavaFX `Scene`. Which of the following represents a correct way to do this?
    a) Scene sc = new Scene(`btn1`, `btn2`, `btn3`);
    b) Scene sc = new Scene(); sc.getChildren().add(`btn1`, `btn2`, `btn3`);
    c) Scene sc = new Scene(new HBox(`btn1`), new VBox(`btn2`), new StackPane(`btn3`));
    d) Scene sc = new Scene(new HBox(new HBox(`btn1`), new VBox(`btn2`, `btn3`)));

EVENT HANDLING

17. What is the **very first phase of event propagation** when a user clicks a Button?
    a) The Bubbling Phase, starting from the Button.
    b) The Handling Phase, where the setOnAction method is called.
    c) The Capturing Phase, starting from the Stage and traveling down towards the Button.
    d) The Consumption Phase, where the OS determines if the event is valid.
18. The **primary purpose** of calling ***event.consume()*** is to:
    a) Mark the event for garbage collection.
    b) Stop the event from propagating further along its current dispatch chain.
    c) Change the event's source to a different node.
    d) Duplicate the event and send it to a different handler.
19. If a **filter on a parent Pane** calls ***event.consume()*** during a MOUSE_CLICKED event, what happens to the child Button that was clicked?
    a) The Button's filters and handlers for MOUSE_CLICKED will not be triggered.
    b) Only the Button's setOnAction handler will be triggered.
    c) The event will still bubble up from the Button.
    d) The Button's handler runs first, so consume() has no effect.
20. What is the critical **difference between** *button.setOnAction(...)* and *button.addEventHandler(ActionEvent.ACTION, ...)*?
    a) *setOnAction* runs during the capturing phase, while *addEventHandler* runs during the bubbling phase.
    b) Calling *setOnAction* a second time replaces the previous handler (i.e. *setOnAction* allows a single handler); *addEventHandler* allows multiple handlers to be registered and coexist.
    c) *setOnAction* can only accept lambda expressions.
    d) *addEventHandler* cannot handle *ActionEvent*.
21. Which event handling implementation style is **most suitable** for creating a reusable handler that needs to maintain its own state (e.g., a counter)?
    a) A single-line lambda expression.
    b) An anonymous inner class.
    c) A separate (named) inner class that implements EventHandler.
    d) A method reference.
22. Examine the following code. What is the output when the button is clicked?

```
Button btn = new Button("Test");
btn.addEventHandler(ActionEvent.ACTION, e -> System.out.print("B"));
btn.setOnAction(e -> System.out.print("A"));
btn.setOnAction(e -> System.out.print("C"));
```

    a) A
    b) B
    c) AB
    d) BA
    e) BC
    f) BAC
23. To validate user input in a TextField *as they type* and prevent invalid characters from ever appearing, the best approach is to use:
    a) A ChangeListener on the textProperty.
    b) An EventFilter for KeyEvent.KEY_TYPED.
    c) An EventHandler for KeyEvent.KEY_PRESSED.
    d) An setOnAction handler.

24. A local variable **count** is declared in a method. Why does the following code fail to compile?

```
int count = 0;
button.setOnAction(e -> {
    count++; // Compilation Error
    System.out.println(count);
});
```

a) The lambda is attempting to modify *count*, which is *not final* or "*effectively final*".
b) Primitives like *int* cannot be accessed from within a lambda.
c) The lambda requires curly braces {} to be a multi-line statement.
d) setOnAction does not permit access to variables outside its scope.

CONTROLS & PROPERTIES

25. To make several **RadioButtons mutually exclusive**, you must:
a) Place them all within the same layout pane.
b) Give them all the same CSS style class.
c) Add them all to the same ToggleGroup object.
d) Manually write listeners to deselect the others.
26. To get the currently selected item from a ***ComboBox<String> cb***, you should use:
a) cb.getText()
b) cb.getSelectionModel().getSelectedItem()
c) cb.getValue()
d) Both b and c are correct and commonly used.
27. The difference between textField.setEditable(false) and textField.setDisable(true) is that:
a) A disabled field cannot be focused and appears greyed out, while a non-editable field can be focused and its text can be selected/copied.
b) There is no functional difference.
c) A non-editable field cannot receive any events.
d) setDisable is for TextArea while setEditable is for TextField.
28. A ProgressBar showing progress = -1.0 is in an:
a) Error state.
b) Indeterminate state.
c) Complete state.
d) Empty state.
29. A ToggleButton is added to a ToggleGroup containing RadioButtons. The effect is that:
a) It will cause a runtime ClassCastException.
b) The ToggleButton will now act like a RadioButton within that group.
c) All RadioButtons will now allow multiple selections.
d) The ToggleGroup will simply ignore the ToggleButton.
30. What is the output of the following code?

```
VBox vbox = new VBox();
Label label = new Label("Initial");
vbox.getChildren().add(label);
label.setText("Changed");
label = new Label("Final");
System.out.println(((Label)vbox.getChildren().get(0)).getText());
```

a) Initial
b) Changed
c) Final
d) The code throws a ClassCastException.
31. Which of the following is considered a "leaf node" in the Scene Graph?
a) GridPane
b) ScrollPane
c) CheckBox
d) BorderPane
32. Regarding CSS styling in JavaFX, which selector has the highest precedence?
a) An ID selector (#my-id)
b) A style class selector (.my-class)
c) An inline style set via node.setStyle(...)
d) A type selector (.button)

33. The primary technical reason that EventHandler<T> can be implemented using a lambda expression is because it:
    a) Is part of the javafx.event package.
    b) Is a functional interface (has only one abstract method).
    c) Always returns void.
    d) Is a generic interface.

Section II: Error Identification and Correction (8 Marks)

Instructions: Each of the following code snippets contains a critical logical error. Identify the error, explain *why* it is an error, and provide the corrected code.

1. Faulty Event Propagation (4 Marks)

Goal: A ***StackPane*** acts as a "root". Clicking anywhere on the root should print "**Root clicked**". However, a Button on the root **should *only*** print "**Button action performed!**" when clicked, not "Root clicked". The current code, given below, prints both messages when the button is clicked. Explain why this is so, then suggest a way to make the button click to display only the message '**Button action performed!**", finally write corrected code.

```
// Inside start() method:
StackPane root = new StackPane();
Button specialButton = new Button("Perform Action");
root.getChildren().add(specialButton);

// Root listener
root.addEventHandler(MouseEvent.MOUSE_CLICKED, e -> {
    System.out.println("Root clicked");
});

// Button listener
specialButton.setOnAction(e -> {
    System.out.println("Button action performed!");
});
```

2. Incorrect Object Reference in UI Logic (4 Marks)

Goal: A utility class **StatusManager** is created to update a **status Label**. The main application class creates an instance of the manager and passes the label to it. However, **clicking the action button does nothing**, and the label's text never changes.

a. Explain why this is so, then suggest a way to make the button click to update the text on status Label to correctly say 'Task Complete!', finally write corrected code (i.e. re-write the full corrected `StatusManager` class again; the '`MainApp`' class has no error).
b. As clearly mentioned, the `MainApp` class has no error. But the setOnAction is created using lambda expression. Convert the line '`actionButton.setOnAction(e -> manager.myTask());`' to use anonymous class instead of the lambda expression. (In other words, re-write this line with its anonymous class equivalent instead of using lambda)

```java
// Main Application Class
public class MainApp extends Application {//class start
    @Override
    public void start(Stage primaryStage) {
        VBox root = new VBox(10);
        Label statusLabel = new Label("Initial State");
        Button actionButton = new Button("Run Task");

        StatusManager manager = new StatusManager(statusLabel);
        actionButton.setOnAction(e -> manager.myTask());

        root.getChildren().addAll(statusLabel, actionButton);
        primaryStage.setScene(new Scene(root, 200, 100));
        primaryStage.show();
    }
}//class End

// Separate Utility Class
class StatusManager { //Utility class start
    private Label statusLabel;

    public StatusManager(Label label) {
        this.statusLabel = new Label("Manager Initialized");
    }

    public void myTask() {
        this.statusLabel.setText("Task Complete!");
    }
}//Utility class End
```

Section III: Short Answer Questions (15 Marks)

(Answer the following questions concisely. Each question carries 6 marks.)

1. Application Lifecycle Question:

You are designing a JavaFX application that must:

- Load resources and create background services
- Display the user interface (Panes, Buttons, etc.)
- Connect to and close a database

Answer the following questions:

a. **Lifecycle Order (2 Marks)**

From the list of method names below, identify which ones are **correct JavaFx lifecycle methods** and **list these correct methods in proper order** they are called from **the first** to **the last**, using arrows (→) between them. (`E.g. A() -> B() -> C() etc.`)

Method list: `main()`, `create()`, `resume()`, `launch()`, `start()`, `config()`, `init()`, `release()`, `run()`, `exit()`, `clean()`, `stop()`

b. **Thread Context (2 Marks)**

For each lifecycle method you listed in Question 1, specify **which thread it runs on**: **main thread** or **JavaFX Application Thread**. Example format: `methodName() – threadName`

c. **Task Assignment (2 Marks)**

Pair the following tasks with the **most appropriate lifecycle method** in your list:

- Load resources and create background services
- Create Panes, Buttons, and the Scene Graph
- Close database connection and clean up resources

2. Layout Question (3Pts):

You have the following VBox defined.

```
VBox vbx=new VBox(…);
```

Write lines of code:

a. To set the padding as follows (bottom:5, top: 10, left: 15, right: 20)

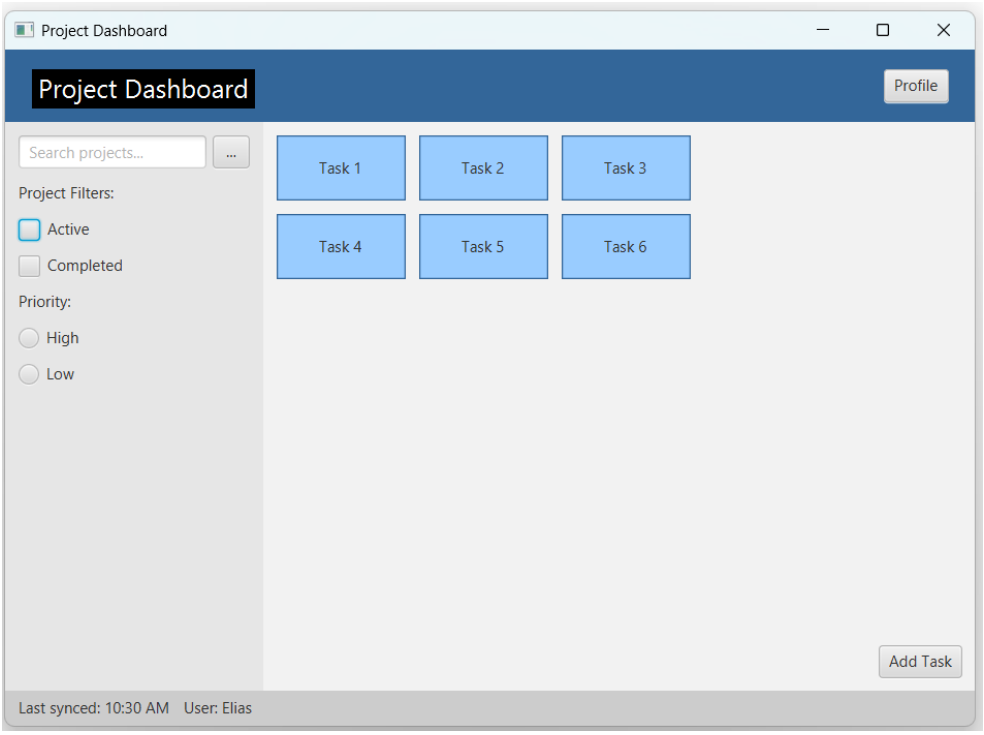
b. To align children of the VBox to center-left

3. UI Analysis (6 Marks)
Instructions:

You are provided with a **screenshot of a JavaFX Project Dashboard UI**. Using the image, analyze the layout and fill in the table below:

For each section of the UI, indicate:

1. The **JavaFX pane used** (e.g., BorderPane, VBox, HBox, GridPane, StackPane, AnchorPane).
2. The **nested panes or controls inside it**, described in terms of **how they are structured**, do not simply list the controls. Describe the nesting clearly focusing on **hierarchy and layout relationships** (e.g., which panes contain which other panes or controls, and how they are arranged).



UI Section (Based on Image)	Pane Used	Nested Panes / Layout Structure
Root Node		
Top Header		
Left Panel		
Center Dashboard		
Bottom Status Bar		

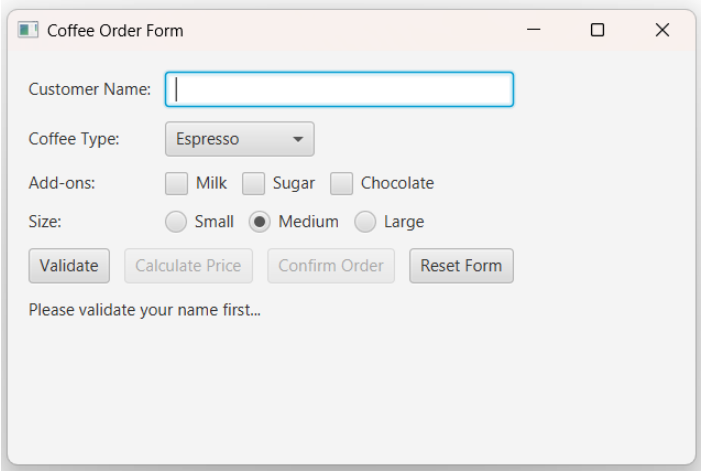
Section IV: Final Comprehensive Coding Challenge (14 Marks)

(Instructions: Read the scenario and requirements carefully. Write a single, complete, and self-contained JavaFX application that implements the solution. Your code will be evaluated on correctness, adherence to all requirements, code clarity, and the proper implementation of the three specified event handling styles.)

Scenario – Coffee Order Form

Instructions:

1. You are provided with an image of the **Coffee Order Form UI**. The UI contains fields for customer name, coffee type, add-ons, size, buttons for actions, and a status label.
2. Your task is to **write a complete JavaFX application** that:
   - Creates the **UI shown in the image**.
   - Implements **event-driven workflow** as described below.
3. You must **write all code yourself** in the skeleton provided. You are not required to exactly match variable names in the UI image, but the structure and workflow must be followed.
4. Focus on **event handling, state management, and correct logic**, not the visual styling.



Workflow Description

1. **Validate Button**:
   - Checks that the customer name is filled.
   - Updates the status label with an appropriate message. E.g. "Name is required!"
   - Enables the next step (calculate price button) if valid.
2. **Calculate Price Button**:
   - Computes the total based on selected coffee size and add-ons.
   - Updates the status label with the calculated total. E.g. "Total: 110.00"
   - Enables the next step (confirm order button) if calculation succeeds.
3. **Confirm Order Button**:
   - Displays a final message in the status label confirming the order with customer name and total. E.g. "Order for Daniel Confirmed! Total: 110.00"
4. **Reset Form Button**:
   - Clears all inputs, resets selections, and restores buttons to initial state.

Additional Notes

- You are free to use any layout (VBox, HBox, GridPane) to construct the UI as shown in the image.
- Ensure **buttons are enabled or disabled correctly** according to the workflow.
- You must **handle invalid inputs**, such as empty name or incorrect quantity if applicable.
- **Do not copy pre-written event handlers**; think about **different ways events can be handled**.
- Focus on **workflow correctness** and **status label updates**.

Ready-to-Use Skeleton

```
import javafx.application.Application;
import javafx.event.ActionEvent;
import javafx.event.EventHandler;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.*;
import javafx.stage.Stage;

public class CoffeeOrderFormSkeleton extends Application {

    // ------------------ UI Controls ------------------
    TextField nameField;
    ComboBox<String> coffeeCombo;
    CheckBox milkCB, sugarCB, chocolateCB;
    RadioButton smallRB, mediumRB, largeRB;
    Button validateBtn, calculateBtn, confirmBtn, resetBtn;
    Label statusLabel;

    // Last calculated total price
    double lastTotal = 0;

    @Override
    public void start(Stage primaryStage) {
        primaryStage.setTitle("Coffee Order Form");

        // ------------------ Layout ------------------
        GridPane grid = new GridPane();
        // TODO: Configure padding, spacing, etc.

        // ------------------ Controls Initialization ------------------
        // TODO: Initialize all controls (TextField, ComboBox, CheckBoxes, RadioButtons)
        // TODO: Add them to the grid layout following the UI image
        // TODO: Create HBox or VBox for buttons if needed

        // ------------------ Event Handlers ------------------

        // 1. Validate Button
        // TODO: Set event handler that checks name and updates statusLabel
        validateBtn = new Button("Validate");
        // validateBtn.setOnAction(...);

        // 2. Calculate Price Button
        // TODO: Set event handler that computes total price and updates statusLabel
        calculateBtn = new Button("Calculate Price");
        // calculateBtn.setOnAction(...);

        // 3. Confirm Order Button
        // TODO: Set event handler that displays final confirmation message
        confirmBtn = new Button("Confirm Order");
        // confirmBtn.setOnAction(...);

        // 4. Reset Form Button
        // TODO: Set event handler that clears inputs and restores initial state
        resetBtn = new Button("Reset Form");
        // resetBtn.setOnAction(...);

        // ------------------ Initial State ------------------
```

```
        // TODO: Disable buttons as necessary and set initial status message
        statusLabel = new Label("Please validate your name first...");

        // Add buttons and label to layout
        HBox buttonBox = new HBox(10, validateBtn, calculateBtn, confirmBtn, resetBtn);
        grid.add(buttonBox, 0, 5, 2, 1);  // Adjust row index as needed
        grid.add(statusLabel, 0, 6, 2, 1);

        // ------------------ Scene ------------------
        Scene scene = new Scene(grid, 500, 300);
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    // ------------------ Helper Methods ------------------
    // TODO: Implement calculation logic
    private void calculateTotalPrice() {
        // Compute base price + add-ons, update lastTotal
        // Update statusLabel
    }

    private void resetForm() {
        // Reset all inputs to default values
        // Disable/enable buttons appropriately
        // Clear statusLabel if needed
        lastTotal = 0;
    }

    // Optionally, you can define inner classes for event handlers if needed

    public static void main(String[] args) {
        launch(args);
    }
}
```

ANSWER SHEET ID: _____

Part-1: MCQ

1.	2.	3.	4.	5.	6.	7.
8.	9.	10.	11.	12.	13.	14.
15.	16.	17.	18.	19.	20.	21.
22.	23.	24.	25.	26.	27.	28.
29.	30.	31.	32.	33.		

Part-2: Errors

1. ...

2. …

Part-3: Short Answers

1. ..
    a. ..
    b. ..
    c. ..
2. ..
    a. ..
    b. ..
3. ..

UI Section (Based on Image)	Pane Used	Nested Panes / Layout Structure
Root Node		
Top Header		
Left Panel		
Center Dashboard		
Bottom Status Bar		